RAJALAKSHMI ENGINEERING COLLEGE

*Approved by AICTE | Affiliated to Anna University | Accredited by NAAC*

**(An Autonomous Institution, Affiliated to Anna University, Chennai)**

**Department of Artificial Intelligence and Data Science**

**V Semester B. Tech AIDS**

## AD23532 – Principles of Data Science

**Academic Year:** 2026-2027 **Batch:** 2024-2028

Name: PRATHOSH

# Ex 3b: Model Planning and Building – Regression

**Register Number:** 241801211 **Date:** 02-10-2026

**Dataset:** Advertising Dataset (Kaggle) **Tools:** Python, Pandas, scikit-learn, Matplotlib, Seaborn

## Aim

To plan, build and evaluate regression models (Linear Regression, Polynomial Regression and Support Vector Regression) that predict product sales from TV, Radio and Newspaper advertising spend, identify the model with the lowest prediction error, and use it to recommend how the advertising budget can be optimised.

## Task 1: Load the Advertising dataset (Kaggle) into a Pandas DataFrame

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.svm import SVR
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')

df = pd.read_csv('advertising.csv')
print("Dataset loaded successfully")
```

Out [1]:

```
Dataset loaded successfully
```

## Task 2: Explore the dataset

### First five records

In [2]:

```
df.head()
```

Out [2]:

| | TV | Radio | Newspaper | Sales |
|---|---|---|---|---|
| **0** | 230.1 | 37.8 | 69.2 | 22.1 |
| **1** | 44.5 | 39.3 | 45.1 | 10.4 |
| **2** | 17.2 | 45.9 | 69.3 | 12.0 |
| **3** | 151.5 | 41.3 | 58.5 | 16.5 |
| **4** | 180.8 | 10.8 | 58.4 | 17.9 |

5 rows × 4 columns

## Last five records

In [3]:

```
df.tail()
```

Out [3]:

| | TV | Radio | Newspaper | Sales |
|---|---|---|---|---|
| **195** | 38.2 | 3.7 | 13.8 | 7.6 |
| **196** | 94.2 | 4.9 | 8.1 | 14.0 |
| **197** | 177.0 | 9.3 | 6.4 | 14.8 |
| **198** | 283.6 | 42.0 | 66.2 | 25.5 |
| **199** | 232.1 | 8.6 | 8.7 | 18.4 |

5 rows × 4 columns

## Shape and data types

In [4]:

```
print("Shape of the dataset:", df.shape)
print("\nData types:")
print(df.dtypes)
```

Out [4]:

```
Shape of the dataset: (200, 4)

Data types:
TV           float64
Radio        float64
Newspaper    float64
Sales  float64 dtype:
object
```

## Missing values and duplicate records

In [5]:

```
print("Missing values in each column:")
print(df.isnull().sum())
print("\nTotal missing values :", df.isnull().sum().sum())
print("Duplicate records    :", df.duplicated().sum())
```

Out [5]:

```
Missing values in each column:
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

Total missing values : 0
Duplicate records    : 0
```

### Summary statistics

In [6]:

```
df.describe().round(2)
```

Out [6]:

| | TV | Radio | Newspaper | Sales |
|---|---|---|---|---|
| **count** | 200.00 | 200.00 | 200.00 | 200.00 |
| **mean** | 147.04 | 23.26 | 30.55 | 15.13 |
| **std** | 85.85 | 14.85 | 21.78 | 5.28 |
| **min** | 0.70 | 0.00 | 0.30 | 1.60 |
| **25%** | 74.38 | 9.98 | 12.75 | 11.00 |
| **50%** | 149.75 | 22.90 | 25.75 | 16.00 |
| **75%** | 218.82 | 36.52 | 45.10 | 19.05 |
| **max** | 296.40 | 49.60 | 114.00 | 27.00 |

8 rows × 4 columns

> **Observation:** The dataset has 200 records and 4 numerical columns with no missing values or duplicates. Advertising spend is in thousands of dollars and Sales is in thousands of units. TV has by far the largest budgets (mean 147.04), followed by Newspaper (30.55) and Radio (23.26).

## Task 3: Select the input features (X) and target variable (y)

In [7]:

```
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

print("Input features (X):", X.columns.tolist(), "| shape:", X.shape)
print("Target variable (y):", y.name, "| shape:", y.shape)
```

Out [7]:

```
Input features (X): ['TV', 'Radio', 'Newspaper'] | shape: (200, 3)
Target variable (y): Sales | shape: (200,)
```

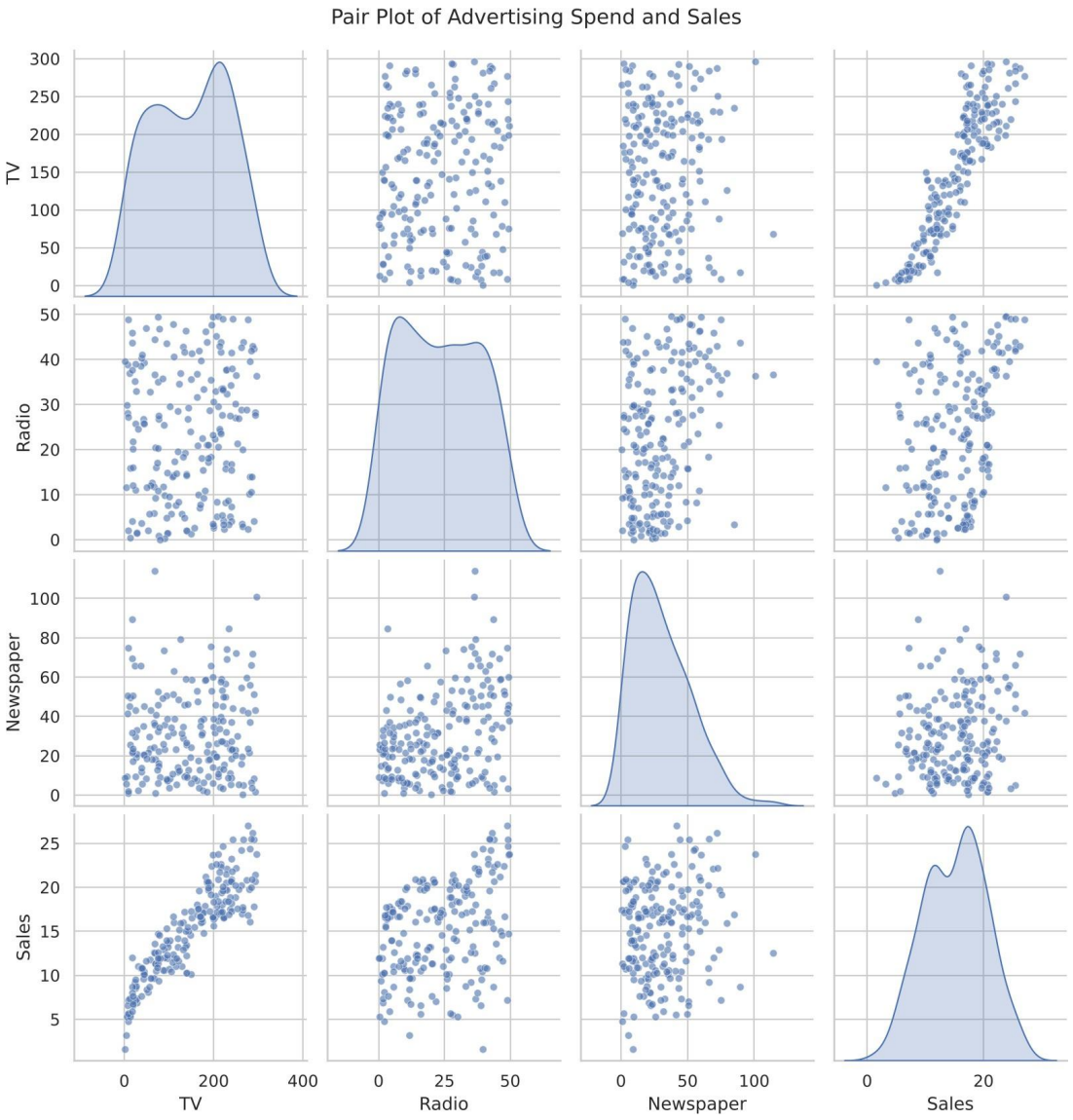
## Exploratory Data Analysis

### Task 4: Pair Plot

In [8]:

```
sns.pairplot(df, vars=['TV', 'Radio', 'Newspaper', 'Sales'], diag_kind='kde',
             plot_kws={'alpha': 0.6, 's': 25, 'color': '#4C72B0'},
             diag_kws={'color': '#4C72B0', 'fill': True}, height=2.6)
plt.suptitle('Pair Plot of Advertising Spend and Sales', y=1.02, fontsize=14)
plt.show()
```

Out [8]:




**Observation:** Sales rises almost in a straight line with TV spend, with the spread of sales widening as TV spend grows. Radio shows a weaker upward trend, and Newspaper shows no clear pattern with Sales. The three advertising channels are not strongly related to each other.

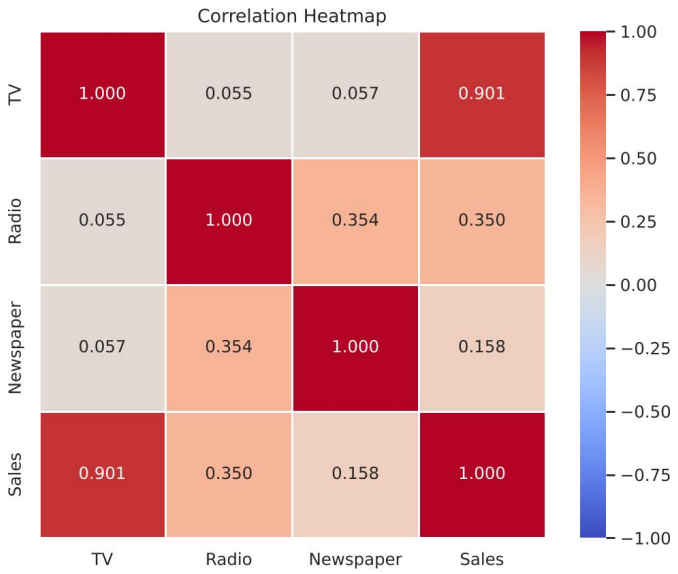
### Task 5: Correlation Heatmap

In [9]:

```
corr = df.corr()

plt.figure(figsize=(7, 5.5))
sns.heatmap(corr, annot=True, fmt='.3f', cmap='coolwarm', vmin=-1, vmax=1, center=0,
            linewidths=0.5, square=True, annot_kws={'size': 11})
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Out [9]:




### Task 6: Feature most strongly correlated with Sales

In [10]:

```
sales_corr = corr['Sales'].drop('Sales').sort_values(ascending=False)
print("Correlation of each feature with Sales:")
print(sales_corr.round(3))
print(f"\nFeature most strongly correlated with Sales: {sales_corr.idxmax()} "
      f"(r = {sales_corr.max():.3f})")
```

Out [10]:

```
Correlation of each feature with Sales:
TV           0.901
Radio        0.350
Newspaper    0.158
Name: Sales, dtype: float64

Feature most strongly correlated with Sales: TV (r = 0.901)
```

## Model Planning and Model Building

### Task 7: Split the dataset into Training (80%) and Testing (20%) sets

In [11]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set:", X_train.shape, "| Testing set:", X_test.shape)
print(f"Training = {len(X_train) / len(X):.0%}, Testing = {len(X_test) / len(X):.0%}")
```

Out [11]:

```
Training set: (160, 3) | Testing set: (40, 3)
Training = 80%, Testing = 20%
```

## Task 8: Linear Regression

In [12]:

```
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

y_pred_lin = lin_reg.predict(X_test)

print("Linear Regression trained successfully")
print("Intercept:", round(lin_reg.intercept_, 4))
for feature, coef in zip(X.columns, lin_reg.coef_):
    print(f"Coefficient of {feature:9s}: {coef:.4f}")
print("\nTraining R2:", round(lin_reg.score(X_train, y_train), 4))
print("Predicted Sales (first 10):", np.round(y_pred_lin[:10], 2))
print("Actual Sales    (first 10):", y_test.values[:10])
```

Out [12]:

```
Linear Regression trained successfully
Intercept: 4.7141
Coefficient of TV    :  0.0545
Coefficient of Radio   :  0.1009
Coefficient of Newspaper: 0.0043

Training R2: 0.9001
Predicted Sales (first 10): [17.03 20.41 23.72  9.27 21.68 12.57 21.08  8.69 17.24 16.67]
Actual Sales    (first 10): [16.9 22.4 21.4  7.3 24.7 12.6 22.3  8.4 16.5 16.1]
```

> **Observation:** Ad spend is in thousands of dollars and Sales in thousands of units. Keeping the other channels fixed, every extra $1,000 adds about 0.0545 thousand units of sales for TV, 0.1009 for Radio and only 0.0043 for Newspaper, so newspaper spending adds almost nothing.

## Task 9: Polynomial Regression (Degree = 2)

In [13]:

```
poly_reg = Pipeline([
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('linear', LinearRegression())
])
poly_reg.fit(X_train, y_train)

y_pred_poly = poly_reg.predict(X_test)

poly_features = poly_reg.named_steps['poly'].get_feature_names_out(X.columns)
print("Polynomial Regression (degree 2) trained successfully")
print("Number of features after expansion:", len(poly_features))
print("Features:", list(poly_features))
print("\nTraining R2:", round(poly_reg.score(X_train, y_train), 4))
print("Predicted Sales (first 10):", np.round(y_pred_poly[:10], 2))
```

Out [13]:

```
Polynomial Regression (degree 2) trained successfully
Number of features after expansion: 9
Features: ['TV', 'Radio', 'Newspaper', 'TV^2', 'TV Radio', 'TV Newspaper', 'Radio^2', 'Radio Newspaper',
'Newspaper^2']

Training R2: 0.9288
Predicted Sales (first 10): [17.56 21.96 22.08  7.35 23.24 12.37 22.13  8.77 17.01 16.91]
```

### Task 10: Support Vector Regression (RBF kernel)

SVR is sensitive to the scale of the inputs, so the features are standardised first. Its hyperparameters (C, gamma, epsilon) are chosen with 5-fold cross-validation on the **training set only**.

In [14]:

```
svr_pipeline =
    Pipeline([ ('scaler',
    StandardScaler()), ('svr',
    SVR(kernel='rbf'))
])

param_grid = {'svr__C': [1, 10, 100, 1000, 10000],
              'svr__gamma': ['scale', 0.001, 0.01, 0.1, 1],
              'svr__epsilon': [0.1, 0.5, 1.0]}

grid = GridSearchCV(svr_pipeline, param_grid, cv=5, scoring='r2')
grid.fit(X_train, y_train)

svr_reg = grid.best_estimator_
y_pred_svr = svr_reg.predict(X_test)

print("SVR (RBF kernel) trained successfully")
print("Best parameters      :", grid.best_params_)
print("Cross-validation R2  :", round(grid.best_score_, 4))
print("Training R2          :", round(svr_reg.score(X_train, y_train), 4))
print("Predicted Sales (first 10):", np.round(y_pred_svr[:10], 2))
```

Out [14]:

```
SVR (RBF kernel) trained successfully
Best parameters      : {'svr__C': 1000, 'svr__epsilon': 1.0, 'svr__gamma': 0.01}
Cross-validation R2  : 0.9097
Training R2          : 0.9287
Predicted Sales (first 10): [17.29 21.52 22.43  7.54 22.94 12.07 22.04  8.8  16.95 16.81]
```

## Model Evaluation

### Task 11: MAE, MSE, RMSE and R² Score

In [15]:

```
predictions = {'Linear Regression': y_pred_lin,
               'Polynomial Regression': y_pred_poly,
               'SVR (RBF)': y_pred_svr}

results = []
for name, y_pred in predictions.items():
    mse = mean_squared_error(y_test, y_pred)
    results.append({'Model': name,
                    'MAE': mean_absolute_error(y_test, y_pred),
                    'MSE': mse,
                    'RMSE': np.sqrt(mse),
                    'R2 Score': r2_score(y_test, y_pred)})

results_df = pd.DataFrame(results).set_index('Model').round(4)
results_df
```

Out [15]:

| Model | MAE | MSE | RMSE | R2 Score |
|---|---|---|---|---|
| **Linear Regression** | 1.2748 | 2.9078 | 1.7052 | 0.9059 |
| **Polynomial Regression** | 0.9034 | 1.4425 | 1.2011 | 0.9533 |
| **SVR (RBF)** | 0.9592 | 1.5704 | 1.2532 | 0.9492 |

3 rows × 4 columns

**Observation:** Polynomial Regression has the lowest errors (MAE 0.9034, RMSE 1.2011) and the highest $R^2$ (0.9533). SVR is a close second (RMSE 1.2532), while Linear Regression has the largest error (RMSE 1.7052) because the effect of advertising on sales is not purely linear.

### Task 12: Comparison of all models

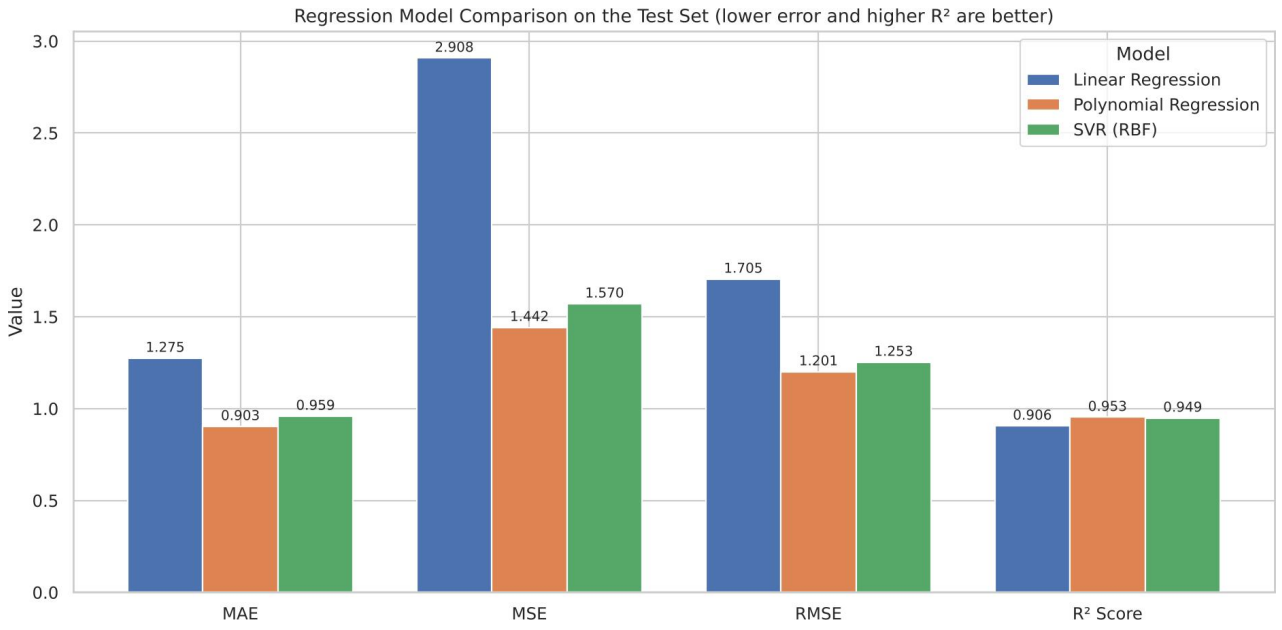
In [16]:

```
metrics = ['MAE', 'MSE', 'RMSE', 'R2 Score']
model_colors = ['#4C72B0', '#DD8452', '#55A868']
x = np.arange(len(metrics))
width = 0.26

fig, ax = plt.subplots(figsize=(12, 6))
for i, (model, color) in enumerate(zip(results_df.index, model_colors)):
    bars = ax.bar(x + (i - 1) * width, results_df.loc[model, metrics], width,
                  label=model, color=color)
    ax.bar_label(bars, fmt='%.3f', fontsize=9, padding=2)

ax.set_xticks(x)
ax.set_xticklabels(['MAE', 'MSE', 'RMSE', 'R² Score'])
ax.set_ylabel('Value')
ax.set_title('Regression Model Comparison on the Test Set (lower error and higher R² are better)')
ax.legend(title='Model')
plt.tight_layout()
plt.show()
```

Out [16]:



## Task 13: Actual vs Predicted Sales for the best-performing model

In [17]:

```
best_model_name = results_df['RMSE'].idxmin()
best_pred = predictions[best_model_name]
print("Best-performing model (lowest RMSE):", best_model_name)

fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

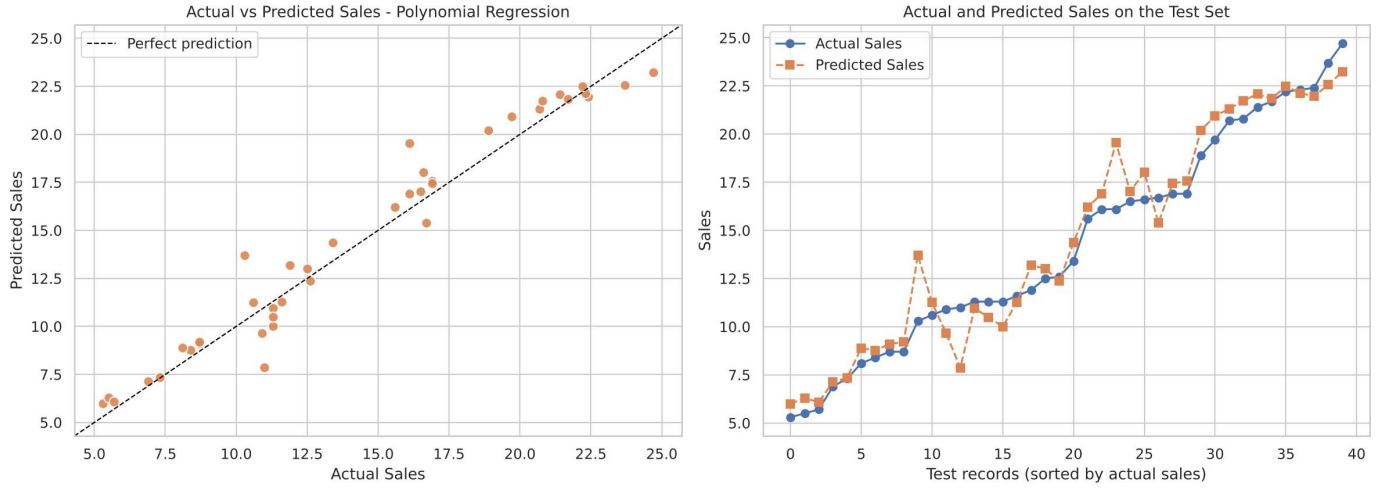
# Scatter plot: actual vs predicted
axes[0].scatter(y_test, best_pred, color='#DD8452', edgecolor='white', s=60, alpha=0.9)
limits = [min(y_test.min(), best_pred.min()) - 1, max(y_test.max(), best_pred.max()) + 1]
axes[0].plot(limits, limits, color='black', linestyle='--', linewidth=1, label='Perfect prediction')
axes[0].set_xlim(limits)
axes[0].set_ylim(limits)
axes[0].set_xlabel('Actual Sales')
axes[0].set_ylabel('Predicted Sales')
axes[0].set_title(f'Actual vs Predicted Sales - {best_model_name}')
axes[0].legend()

# Line plot: actual and predicted sales for each test record
order = np.argsort(y_test.values)
axes[1].plot(y_test.values[order], marker='o', color='#4C72B0', label='Actual Sales')
axes[1].plot(best_pred[order], marker='s', color='#DD8452', linestyle='--', label='Predicted Sales')
axes[1].set_xlabel('Test records (sorted by actual sales)')
axes[1].set_ylabel('Sales')
axes[1].set_title('Actual and Predicted Sales on the Test Set')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Out [17]:

```
Best-performing model (lowest RMSE): Polynomial Regression
```




> **Observation:** The predicted values lie close to the perfect-prediction line across the whole range of sales, so the Polynomial Regression model predicts both low and high sales accurately.

## Model Interpretation

### Task 14: Model with the lowest prediction error

In [18]:

```
print("Lowest MAE  :", results_df['MAE'].idxmin(), "->", results_df['MAE'].min())
print("Lowest MSE  :", results_df['MSE'].idxmin(), "->", results_df['MSE'].min())
print("Lowest RMSE :", results_df['RMSE'].idxmin(), "->", results_df['RMSE'].min())
print("Highest R2  :", results_df['R2 Score'].idxmax(), "->", results_df['R2 Score'].max())
```

Out [18]:

```
Lowest MAE  : Polynomial Regression -> 0.9034
Lowest MSE  : Polynomial Regression -> 1.4425
Lowest RMSE : Polynomial Regression -> 1.2011
Highest R2  : Polynomial Regression -> 0.9533
```

In [19]:

```
# Coefficients of the polynomial model show how the channels affect sales
poly_coef = pd.Series(poly_reg.named_steps['linear'].coef_, index=poly_features)
print("Polynomial Regression coefficients:")
print(poly_coef.round(6))
```

Out [19]:

```
Polynomial Regression coefficients:
TV                 0.076216
Radio             -0.031984
Newspaper         -0.001920
TV^2              -0.000106
TV Radio           0.000419
TV Newspaper      -0.000026
Radio^2            0.001448
Radio Newspaper    0.000165
Newspaper^2        0.000085
dtype: float64
```

> **Answer: Polynomial Regression (degree 2)** has the lowest prediction error on every metric: MAE 0.9034, MSE 1.4425, RMSE 1.2011 and $R^2$ 0.9533, so it explains about 95% of the variation in sales. Its positive TV × Radio coefficient means TV and radio advertising work better together than separately, and its negative $TV^2$ coefficient shows diminishing returns at very high TV budgets.

### Budget scenarios using the best model

The best model is used to compare three ways of spending the same total budget (about $200.9k, the average total spend in the dataset). All values stay within the range of the training data.

In [20]:

```
scenarios = pd.DataFrame({
    'TV':        [147.0, 147.0, 172.0],
    'Radio':     [ 23.3,  48.3,  23.3],
    'Newspaper': [ 30.6,   5.6,   5.6]},
    index=['A: Current average mix',
           'B: $25k Newspaper -> Radio',
           'C: $25k Newspaper -> TV'])

scenarios['Total Budget'] = scenarios[['TV', 'Radio', 'Newspaper']].sum(axis=1)
scenarios['Predicted Sales'] = poly_reg.predict(scenarios[['TV', 'Radio', 'Newspaper']]).round(2)
scenarios
```

Out [20]:

| | TV | Radio | Newspaper | Total Budget | Predicted Sales |
|---|---|---|---|---|---|
| **A: Current average mix** | 147.0 | 23.3 | 30.6 | 200.9 | 15.57 |
| **B: $25k Newspaper -> Radio** | 147.0 | 48.3 | 5.6 | 200.9 | 18.89 |
| **C: $25k Newspaper -> TV** | 172.0 | 23.3 | 5.6 | 200.9 | 16.84 |

3 rows × 5 columns

> **Observation:** With the same total budget, moving $25k from Newspaper to Radio raises predicted sales from 15.57 to 18.89 thousand units (about 21% more), while moving it to TV raises sales to 16.84 thousand units (about 8% more).

### Task 15: Recommendations to optimise the advertising budget

1. **Move newspaper spending to radio.** Newspaper has the weakest link with sales (r = 0.158) and adds almost nothing per $1,000 in the linear model. In the best model, moving $25k from newspaper to radio increases predicted sales by about 21% at the same total budget.
2. **Keep TV as the main channel, but watch for diminishing returns.** TV has the strongest correlation with sales (r = 0.901). However, the negative $TV^2$ term shows that each extra dollar adds less at very high TV budgets, so further money is better spent on radio once TV spend is already high.
3. **Run TV and radio campaigns together and plan budgets with the model.** The positive TV × Radio interaction means combined campaigns produce more sales than either channel alone. The company can use the Polynomial Regression model to test budget plans before spending, and retrain it as new campaign data comes in.

## Result

Thus Linear Regression, Polynomial Regression (degree 2) and SVR (RBF kernel) models were built using Python to predict sales from advertising spend and evaluated using MAE, MSE, RMSE and $R^2$ score. TV was the feature most strongly correlated with sales (r = 0.901). Polynomial Regression had the lowest prediction error (RMSE 1.2011, $R^2$ 0.9533), and based on this model three recommendations were made to optimise the advertising budget.